

Questopm 1

Consider this summation $\sum_{i=x}^y i^2$.

Part A

Determine the equivalent closed-form expression of this summation. (In this context, 'closed-form' means 'not a summation of $y - x + 1$ terms.')

Include a short explanation of how you created the expression.

The closed-form of the summation $\sum_{i=x}^y i^2$ is:

$$\frac{1}{3}y^3 + \frac{1}{2}y^2 + \frac{1}{6}y - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right)$$

To derive this expression, we start by noting that a sum of powers such as $\sum_{i=1}^n i^2$ is typically a polynomial of degree higher than the power of the summand (in this case, i^2 implies a polynomial of degree 3). Assume the form:

$$\sum_{i=1}^y i^2 = a_3y^3 + a_2y^2 + a_1y + a_0$$

For now assume $x = 1$ and plug in small values of y .

$$\sum_{i=1}^1 i^2 = a_3(1)^3 + a_2(1)^2 + a_1(1) + a_0 = a_3 + a_2 + a_1 + a_0 = 1$$

$$\sum_{i=1}^2 i^2 = a_3(2)^3 + a_2(2)^2 + a_1(2) + a_0 = 5a_3 + 4a_2 + 2a_1 + a_0 = 5$$

$$\sum_{i=1}^3 i^2 = a_3(3)^3 + a_2(3)^2 + a_1(3) + a_0 = 14a_3 + 9a_2 + 3a_1 + a_0 = 14$$

$$\sum_{i=1}^4 i^2 = a_3(4)^3 + a_2(4)^2 + a_1(4) + a_0 = 30a_3 + 16a_2 + 4a_1 + a_0 = 30$$

We can find the values of a_3 , a_2 , a_1 , and a_0 by solving the system of equations using matrices. Form the coefficient matrix A and the constant matrix B :

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 8 & 4 & 2 & 1 \\ 27 & 9 & 3 & 1 \\ 64 & 16 & 4 & 1 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 \\ 5 \\ 14 \\ 30 \end{bmatrix}$$

$$AX = B \Rightarrow X = A^{-1}B$$

$$X = \begin{bmatrix} a_3 \\ a_2 \\ a_1 \\ a_0 \end{bmatrix} = A^{-1}B = \begin{bmatrix} \frac{1}{3} \\ \frac{1}{2} \\ \frac{1}{6} \\ 0 \end{bmatrix}$$

Therefore, the closed-form expression of the summation $\sum_{i=1}^y i^2$ is:

$$\frac{1}{3}y^3 + \frac{1}{2}y^2 + \frac{1}{6}y$$

When the lower bound of the summation is x instead of 1, we are essentially subtracting the sum of the first $x - 1$ terms from the sum of the first y terms.

The sum of the first y terms was found previously:

$$S_y = \frac{1}{3}y^3 + \frac{1}{2}y^2 + \frac{1}{6}y$$

The sum of the first $x - 1$ terms is:

$$S_{x-1} = \frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1)$$

Therefore, the sum of the terms from x to y is:

$$S_y - S_{x-1} = \frac{1}{3}y^3 + \frac{1}{2}y^2 + \frac{1}{6}y - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right)$$

Part B

Prove that your answer to (b) is correct

Proof by Induction on $y \geq x$

Base Case: $y = x$

$$\sum_{i=x}^x i^2 = x^2 = \frac{1}{3}x^3 + \frac{1}{2}x^2 + \frac{1}{6}x - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right)$$

$$x^2 = \frac{1}{3}x^3 + \frac{1}{2}x^2 + \frac{1}{6}x - \left(\frac{1}{3}x^3 - \frac{1}{3}x^2 + \frac{1}{2}x^2 - \frac{1}{2}x + \frac{1}{6}x - \frac{1}{6} \right)$$

$$x^2 = x^2$$

Inductive Hypothesis: Assume the statement is true for $y = k$

$$\sum_{i=x}^k i^2 = \frac{1}{3}k^3 + \frac{1}{2}k^2 + \frac{1}{6}k - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right)$$

Inductive Step: Prove the statement is true for $y = k+1$ given the statement is true for $y = k$

$$\text{WTSP } \sum_{i=x}^{k+1} i^2 = \frac{1}{3}(k+1)^3 + \frac{1}{2}(k+1)^2 + \frac{1}{6}(k+1) - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right)$$

$$\sum_{i=x}^{k+1} i^2 = \sum_{i=x}^k i^2 + (k+1)^2$$

Plug in the inductive hypothesis:

$$\frac{1}{3}k^3 + \frac{1}{2}k^2 + \frac{1}{6}k - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right) + (k+1)^2$$

Expand $(k+1)^2$:

$$\frac{1}{3}k^3 + \frac{1}{2}k^2 + \frac{1}{6}k - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right) + k^2 + 2k + 1$$

Combine the k^2 , $2k$, and constant terms:

$$\frac{1}{3}k^3 + \left(1 + \frac{1}{2}\right)k^2 + \left(\frac{1}{6} + 2\right)k + 1 - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right)$$

This simplifies to:

$$\frac{1}{3}(k+1)^3 + \frac{1}{2}(k+1)^2 + \frac{1}{6}(k+1) - \left(\frac{1}{3}(x-1)^3 + \frac{1}{2}(x-1)^2 + \frac{1}{6}(x-1) \right)$$

This is exactly the closed-form expression we derived in Part A. Therefore, if the statement is true for $y = k$, then it is true for $y = k + 1$. By induction, the statement is true for all $y \geq x$. ■

Question 2

Shown below are two pseudocode algorithms. For each algorithm, use step-counting (as demonstrated in class) to produce a polynomial expression for the number of steps performed by the algorithm, in terms of the instance characteristic n . Show your augmented code (with the counter increments and loop iterations, like I did in class).

Part A

```
sum <-- 0
product <-- 1
for i from 1 through 3n do
    product <-- product * i
    sum <-- sum + product
end for
```

Augmented Pseudocode

```
o++;
o++;
o++;
iterate 3n times:
    o++;
    o++;
    o++;
    o++;
```

There are three constant steps:

1. Assigning 0 to `sum`
2. Assigning 1 to `product`
3. Assigning 0 to `i`

Inside the loop, there are 4 steps:

1. Checking the loop condition
2. Assigning `product * i` to `product` (in-place multiplication)
3. Assigning `sum + product` to `sum` (in-place addition)
4. Incrementing `i`

Therefore, the total number of steps is:

$$3 + 4(3n)$$

Part B

```
sum <-- 0
for i from 1 through n*n do
  for j from 1 through i do
    sum <-- sum + i * j
  end for
end for
```

Augmented Pseudocode

```
o++;
iterate n^2 times:
  o++;
  iterate i times:
    o++;
    o++; o++; o++;
    o++;
  o++;
```

There is one constant step:

1. Assigning 0 to `sum`

Inside the outer loop there are two steps (not including the inner loop):

1. Checking the loop condition
2. Incrementing `i`

Inside the inner loop, there are 5 steps:

1. Checking the loop condition
2. Creating a temporary variable to store `i * j`
3. Assigning `i * j` to the temporary variable
4. Assigning `sum + i * j` to `sum` (in-place addition)
5. Incrementing `j`

Therefore, the total number of steps is:

$$1 + 2(n^2) + 5 \left(\sum_{i=1}^{n^2} i \right) = 1 + 2(n^2) + 5 \left(\frac{n^2(n^2 + 1)}{2} \right) = 1 + 2n^2 + \frac{5n^2(n^2 + 1)}{2}$$

Question 3

Arrange the following functions in ascending order by rate of growth (that is, slowest-growing to fastest-growing).

$$\frac{n}{2}, n^2, \log_2 6n, 12, 4^{\frac{n}{2}}, \log_6 2n, \sqrt{n}$$

1. 12 (constant function)
2. $\log_6 2n$ (logarithmic function)
3. $\log_2 6n$ (logarithmic function) (Since $\log_2(6n) = \log_2(6) + \log_2(n)$ and $\log_6(2n) = \frac{1+\log_2(n)}{\log_2(6)}$, $\log_6(2n)$ grows slower than $\log_2(6n)$ due to the factor $\frac{1}{\log_2(6)}$)
4. $\sqrt{n}$ (square root function) ($\sqrt{n}$ grows slower than the linear function $\frac{n}{2}$ because $\lim_{n \rightarrow \infty} \frac{\sqrt{n}}{n/2} = 0$)
5. $\frac{n}{2}$ (linear function)
6. n^2 (quadratic function)
7. $4^{\frac{n}{2}}$ (exponential function) (Since $4^{\frac{n}{2}} = 2^n$, $4^{\frac{n}{2}}$ grows faster than n^2 because $\lim_{n \rightarrow \infty} \frac{2^n}{n^2} = \infty$)

Question 4

Shown below are two time complexity functions. Answer the following question for each of them. See Section 3.3 of the PDF version of Shaffer's text for some background.

Imagine that an algorithm's execution efficiency is described by the function $E()$. You execute that algorithm on Computer A and discover that it takes s seconds to complete a problem of size n . You buy Computer B that is 32 times faster than Computer A. All else being equal, how large of a problem (as an expression in terms of n) can Computer B complete in the same amount of time (s seconds), using the same algorithm?

Part A

$$E(n) = 4n^2$$

The execution time on Computer B for the same problem size n would be $\frac{E(n)}{32}$. We want to find the largest problem size n such that the execution time on Computer B is equal to the execution time on Computer A such that:

For computer A, the time to solve the problem of size n is: $E(n) = cn^2 = 4n^2$.

For Computer B, which is 32 times faster, it can solve a problem of size n_b in the same time, so the relationship is:

$$E(n_b) = 32E(n) \Rightarrow 4n_b^2 = 32(4n^2)$$

Cancel out the 4s:

$$n_b^2 = 32n^2$$

Square root both sides:

$$n_b = \sqrt{32}n = 4\sqrt{2}n$$

Therefore, Computer B can solve a problem of size $4\sqrt{2}n$ in the same time as Computer A can solve a problem of size n .

Part B

$$E(n) = 3^n$$

For Computer A, the time to solve the problem of size n is: $E(n) = 3^n$.

For Computer B, which is 32 times faster, it can solve a problem of size n_b in the same time, so the relationship is:

$$E(n_b) = 32E(n) \Rightarrow 3^{n_b} = 32(3^n)$$

$$3^{n_b} = 3^5 3^n = 3^{n+5}$$

Take the log base 3 of both sides:

$$\log_3 3^{n_b} = \log_3 3^{n+5}$$

$$n_b = n + 5$$

Therefore, Computer B can solve a problem of size $n + 5$ in the same time as Computer A can solve a problem of size n .

Question 5

Using the definition of Big-O presented in class, show that $f(n) = 100$ is $O(1)$ and also $O(n^2)$. (Need the definition? See the slides of Sept. 12th.)

Definition of Big-O:

Let $f(n)$ and $g(n)$ be functions s.t. $(f, g : \mathbb{N} \rightarrow \mathbb{R})$.

We say that $f(n)$ is $O(g(n))$ (or $f(n) \in O(g(n))$) if **there exists** a real constant $c > 0$ and **there exists** an integer constant $n_0 \geq 1$ such that $0 \leq f(n) \leq cg(n)$ for all $n \geq n_0$.

Proof that $f(n) = 100$ is $O(1)$

Let $g(n) = 1$. We need to find a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $0 \leq 100 \leq c$ for all $n \geq n_0$.

Let $c = 100$ and $n_0 = 1$. Then $0 \leq 100 \leq 100$ for all $n \geq 1$. Therefore, $f(n) = 100$ is $O(1)$.

Proof that $f(n) = 100$ is $O(n^2)$

Let $g(n) = n^2$. We need to find a constant $c > 0$ and an integer constant $n_0 \geq 1$ such that $0 \leq 100 \leq cn^2$ for all $n \geq n_0$.

Let $c = 1$. Then, we need $100 \leq n^2$ for all $n \geq n_0$. This is true for all $n \geq 10$. Therefore, $f(n) = 100$ is $O(n^2)$.

Question 6

What is the smallest integer value of c such that $f(n) = \sqrt{n}$ is $O(n^c)$? (We're only asking for the answer, but, as the assignment directions say, show your work for possible partial credit.)

We need to find the smallest integer value of c such that $\sqrt{n} \leq Cn^c$ for all $n \geq n_0$.

$$\sqrt{n} \leq Cn^c$$

Divide both sides by n^c :

$$n^{\frac{1}{2}-c} \leq C$$

For this inequality to hold as n approaches infinity, the exponent of n must be less than or equal to 0 (otherwise it will grow without bound). Therefore:

$$\frac{1}{2} - c \leq 0 \Rightarrow c \geq \frac{1}{2}$$

Since c must be an integer, the smallest integer value of c such that $\sqrt{n}$ is $O(n^c)$ is $c = 1$.

Question 7

Write an $O(n)$ method (in your choice of Java or pseudocode) that accepts references to two doubly-linked lists of integers (both in ascending order) and returns a reference to a new doubly-linked list, also in ascending order, that contains all of the values of the two input lists. Include an explanation of why it is $O(n)$.

```
public static DoublyLinkedList merge(DoublyLinkedList list1, DoublyLinkedList list2) {
    DoublyLinkedList merged = new DoublyLinkedList();
    Node current1 = list1.head;
    Node current2 = list2.head;

    while (current1 != null && current2 != null) {
        if (current1.data <= current2.data) {
            merged.add(current1.data);
            current1 = current1.next;
        } else {
            merged.add(current2.data);
            current2 = current2.next;
        }
    }

    // Add remaining nodes from list1 (if any)
    while (current1 != null) {
        merged.add(current1.data);
        current1 = current1.next;
    }

    // Add remaining nodes from list2 (if any)
    while (current2 != null) {
        merged.add(current2.data);
        current2 = current2.next;
    }

    return merged;
}
```

The time complexity of this method is $O(n)$ where n is the total number of nodes in the two input lists. The method iterates through the two input lists exactly once, adding each node to the merged list. The time complexity is linear with respect to the total number of nodes in the input lists, making it $O(n)$.

Question 8

For each of the pairs of functions given below, is $f(n) \in O(g(n))$, $f(n) \in \Omega(g(n))$, or $f(n) \in \Theta(g(n))$? Use limits to determine your answers (as presented in class, and also in Shaffer Section 3.4.5)

To solve this problem, we first find the limit of the ratio of $f(n)$ to $g(n)$ as n approaches infinity.

- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$
 - $f(n) \in O(g(n))$. $f(n)$ grows slower than $g(n)$ as n approaches infinity.
 - $g(n)$ grows asymptotically faster than $f(n)$, so $f(n)$ is upper-bounded by $g(n)$.
- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$
 - $f(n) \in \Omega(g(n))$. $f(n)$ grows faster than $g(n)$ as n approaches infinity.
 - $f(n)$ eventually overtakes $g(n)$, so $f(n)$ is lower-bounded by $g(n)$.
- If $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = c$ where $0 < c < \infty$
 - $f(n) \in \Theta(g(n))$. $f(n)$ grows at the same rate as $g(n)$ as n approaches infinity.
 - $f(n)$ and $g(n)$ grow at the same rate, so $f(n)$ is both upper and lower-bounded by $g(n)$.

Part A

$$f(n) = 5n, g(n) = \frac{n}{100}$$

Find the limit of the ratio of $f(n)$ to $g(n)$ as n approaches infinity:

$$\lim_{n \rightarrow \infty} \frac{5n}{\frac{n}{100}} = \lim_{n \rightarrow \infty} 500 = 500$$

Since the limit is a constant $c > 0$, $f(n) \in \Theta(g(n))$.

Part B

$$f(n) = \log_2^2 n, g(n) = \log_2 n$$

Find the limit of the ratio of $f(n)$ to $g(n)$ as n approaches infinity:

$$\lim_{n \rightarrow \infty} \frac{\log_2^2 n}{\log_2 n} = \lim_{n \rightarrow \infty} \log_2 n = \infty$$

Since the limit is ∞ , $f(n) \in \Omega(g(n))$.

Question 9

Consider a $\Theta(\log_2 n)$ version of the binary search algorithm that accepts a 1D array of integers (stored in ascending order) plus a search key value, and returns the index of the last occurrence of that key within the array ('last' meaning the occurrence with the largest array index). Create this algorithm, in your choice of Java or pseudocode, and include an explanation of why it is $\Theta(\log_2 n)$.

Be aware: To achieve the required level of performance, linear search cannot be used.

```
public static int binarySearchLastOccurrence(int[] arr, int key) {
    int low = 0;
    int high = arr.length - 1;
    int result = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == key) {
            result = mid;
            low = mid + 1;
        } else if (arr[mid] < key) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }

    return result;
}
```

The time complexity of this algorithm is $\Theta(\log_2 n)$ where n is the number of elements in the input array.

The algorithm uses a binary search approach to find the last occurrence of the search key within the array.

At each step, the algorithm divides the search space in half, reducing the number of elements to search by half. This logarithmic reduction in the search space results in a time complexity of $\Theta(\log_2 n)$.

Question 10

For each of the following recurrence relations, if the Master Theorem can be used, use it to determine its Big- Θ asymptotic solution. If the Master Theorem cannot be used, solve the recurrence (showing your work!) and then determine its Big- Θ asymptotic solution.

To determine if the **Master Theorem** can be used, there are specific conditions that the recurrence relation must satisfy. The Master Theorem can be applied to recurrences of the form:

$$T(n) = aT\left(\frac{n}{b}\right) + c \cdot n^d$$

where:

- $T(n)$ is an increasing function
- $n = b^k$, where k is a non-negative integer
- a is a real and $a \geq 1$
- b is an integer and $b > 1$
- c is a real and $c > 0$
- d is a real and $d \geq 0$

Then:

$$T(n) = \begin{cases} \Theta(n^d) & \text{if } d > \log_b a \\ \Theta(n^d \log n) & \text{if } d = \log_b a \\ \Theta(n^{\log_b a}) & \text{if } d < \log_b a \end{cases}$$

Part A

$F(n) = 4F(n/2) + 2n$, assuming that $F(1) = 1$ and n is a power of 2.

In this case, $a = 4$, $b = 2$, $c = 2$, and $d = 1$. All of the conditions for the Master Theorem are satisfied, so we can apply the theorem:

$$d = 1 < \log_b a = \log_2 4 = 2 \Rightarrow F(n) = \Theta(n^{\log_2 4}) = \Theta(n^2)$$

Part B

$G(i) = 3G(i-3) + k$, assuming that $G(0) = 0$ and i is a multiple of 3.

In the standard form, the problem size decreases by a constant fraction in each step, like n/b where $b > 1$. In this case, the problem size decreases by a constant amount in each step ($i - 3$). Therefore, the Master Theorem cannot be applied.

Instead, we can solve the recurrence relation by expanding it.

Start with the original recurrence relation:

,

$$G(i) = 3G(i - 3) + k$$

Find the $G(i - 3)$ term:

$$G(i - 3) = 3G(i - 6) + k$$

Substitute this back into the original equation:

$$G(i) = 3(3G(i - 6) + k) + k = 9G(i - 6) + 3k + k$$

Find the $G(i - 6)$ term:

$$G(i - 6) = 3G(i - 9) + k$$

Substitute this back into the equation:

$$G(i) = 9(3G(i - 9) + k) + 3k + k = 27G(i - 9) + 9k + 3k + k$$

Find the $G(i - 9)$ term:

$$G(i - 9) = 3G(i - 12) + k$$

Substitute this back into the equation:

$$G(i) = 27(3G(i - 12) + k) + 9k + 3k + k = 81G(i - 12) + 27k + 9k + 3k + k$$

This pattern continues, and we can see that the general form of the solution is:

$$G(i) = 3^k G(i - 3k) + k \sum_{j=0}^{k-1} 3^j$$

$$G(i) = 3^k G(i - 3k) + k \left(\frac{3^k - 1}{2} \right)$$

Since i is a multiple of 3, we can write $i = 3k$:

$$G(3k) = 3^k G(3k - 3k) + k \left(\frac{3^k - 1}{2} \right)$$

$$G(3k) = 3^k G(0) + k \left(\frac{3^k - 1}{2} \right)$$

Since $G(0) = 0$:

$$G(3k) = k \left(\frac{3^k - 1}{2} \right)$$

Therefore, the Big-Theta asymptotic solution is $\Theta(3^{i/3})$.

Part C

$H(j) = H(j/3) + c$, assuming that $H(1) = 3$ and j is a power of 3.

In this case, $a = 1$, $b = 3$, $c = 1$, and $d = 0$. All of the conditions for the Master Theorem are satisfied, so we can apply the theorem:

$$d = 0 = \log_b a = \log_3 1 = 0 \Rightarrow H(j) = \Theta(n^d \log n) = \Theta(\log j)$$

Therefore, the Big-Theta asymptotic solution is $\Theta(\log j)$.